

JAVA INTERVIEW TRAPS.



@java_in.depth

JAVA INTERVIEW TRAPS.

Questions that look simple — but eliminate 80% of candidates. These aren't tricks. They reveal whether you actually understand the JVM.

STRING POOL & THE == LIE

```
String a = "hello";  
String b = "hello";  
String c = new String("hello");  
  
System.out.println(a == b);      // ?  
System.out.println(a == c);      // ?  
System.out.println(a.equals(c)); // ?
```

Most say

"All three print true — strings are the same value."

✓ Correct

true / false / true. `new String()` bypasses the pool — different heap object.

JVM fact: String literals are interned at class-load time via the constant pool (JVM §5.1)

THE INTEGER CACHE

SURPRISE

```
Integer x = 127;  
Integer y = 127;  
Integer p = 128;  
Integer q = 128;  
  
System.out.println(x == y); // ?  
System.out.println(p == q); // ?
```

Most say

"Both false — Integer is an object, always use `.equals()`."

✓ Correct

true / false. JVM caches Integer `-128` to `127` by default.

Why it matters: This exact bug caused a **production incident at a major fintech** — comparison logic worked in tests (small IDs) and silently broke in prod (large IDs). Never use `==` on boxed types.

HASHMAP IN MULTI-THREADED CODE

```
// Used in a shared service bean:  
Map<String, String> cache =  
    new HashMap<>();  
  
// Thread A and Thread B both call:  
cache.put(key, value);  
cache.get(key);
```

Most say

"It'll probably work fine, maybe occasional wrong value."

✓ Correct

Can cause infinite loop / CPU spike. In Java 7, concurrent resizing caused circular linked lists — 100% CPU.

Real data • HashMap's `resize()` is not atomic •
UseConcurrentHashMap always in shared context

FINAL DOESN'T MEAN IMMUTABLE

```
final List<String> list =  
    new ArrayList<>();  
  
list.add("Java"); // allowed?  
list = new ArrayList<>(); // allowed?
```

Most say

"final list is immutable — both lines will fail to compile."

✓ Correct

.add() compiles fine. = reassignment fails. final locks the reference, not the object.

Correct immutability: Use `List.of()` or `Collections.unmodifiableList()`. Java 16+ Records give you true immutability with less boilerplate.

FINALLY THAT SWALLOWS YOUR EXCEPTION

```
try {  
    throw new RuntimeException("original");  
} finally {  
    throw new RuntimeException("finally");  
}  
// Which exception does caller see?
```

Most say

"Both exceptions are thrown / caller sees 'original'."

✓ Correct

Caller sees only "finally". The original exception is silently lost. A debugging nightmare.

JLS §14.20.2 • If finally throws, original exception is discarded — no suppression, just gone

VIRTUAL THREADS **DON'T** FIX EVERYTHING

```
// Java 21 – Virtual Thread
Thread.ofVirtual().start(() -> {
    synchronized (sharedLock) {
        doSlowDbQuery(); // blocks here
    }
});
// Problem?
```

Most say

"Virtual threads are lightweight — no issue, JVM handles it."

✓ Correct

Thread pinning. `synchronized` pins the virtual thread to its carrier OS thread, killing scalability.

2026 reality: JDK 24 reduces pinning with `synchronized` reform (JEP 491), but knowing why it happened is what separates candidates. Use **ReentrantLock** in virtual-thread-heavy code.

6 TRAPS.

DON'T GET CAUGHT.

- `1String` pool — `new String()` bypasses interning
- Integer cache — `==` works until 127, then silently breaks
- `HashMap` — concurrent writes = potential infinite loop
- `final` $\neq$ `immutable` — locks reference, not content
- `finally throw` — swallows the original exception
- `Virtual threads` + `synchronized` = thread pinning

SAVE THIS POST

Follow for more →

Comment "java" for pdf